Edge cases (an unhandled routing value) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `conditional_routing_practice.py` (Edge cases section): a conditional edge's routing mapping doesn't have to cover every value its function could return — if the function returns a key that's missing from the mapping, LangGraph raises a `KeyError` with no obvious connection to your code. This exercise triggers that on purpose, so it fails loudly and recognizably, before you ever see it happen by accident in a real graph. **If not:** the first unmapped routing value in a real graph would be a mystery `KeyError` several stack frames deep inside LangGraph, with no smaller example telling you what it means.

Solution

```
# conditional_routing_practice.py — Edge cases section
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class GraphState(TypedDict):
    flag: bool | None
    message: str

def node_decide(state: GraphState) -> dict:
    return {}

def node_a(state: GraphState) -> dict:
    return {"message": "path A"}

def node_b(state: GraphState) -> dict:
    return {"message": "path B"}

def route(state: GraphState) -> str:
    if state["flag"] is None:
        return "path_c"
    if state["flag"]:
        return "path_a"
    return "path_b"
```

```
builder = StateGraph(GraphState)
builder.add_node("node_decide", node_decide)
builder.add_node("node_a", node_a)
builder.add_node("node_b", node_b)
builder.add_edge(START, "node_decide")
builder.add_conditional_edges(
    "node_decide", route, {"path_a": "node_a", "path_b": "node_b"},
)
builder.add_edge("node_a", END)
builder.add_edge("node_b", END)

# why: compiling never runs route() -- it only registers graph structure
graph = builder.compile()

test_cases = [True, False, None]
for flag_value in test_cases:
    try:
        result = graph.invoke({"flag": flag_value, "message": ""})
        print(f"flag={flag_value}: OK -> {result['message']}")
    except KeyError as e:
        print(f"flag={flag_value}: FAILED LOUDLY -> KeyError({e})")
```

Expected output:

```
flag=True: OK -> path A
flag=False: OK -> path B
flag=None: FAILED LOUDLY -> KeyError('path_c')
```